32-step Sequence Monophonic Synthetizer Written in Assembly on an ATmega328P

Microcontroller Lab Project Report

Lucas Placentino
lucas.placentino@vub.be

1 INTRODUCTION

This project is a 32-step monophonic synthesizer and sequencer implemented entirely in bare-metal AVR assembly on an ATmega328P microcontroller. It allows users to program a sequence of musical notes, adjust playback speed (in BPM) dynamically.The system utilizes multiple hardware peripherals, including an LED matrix display, an analog joystick, a 4x4 matrix keypad, a piezo buzzer, and the microcontroller's internal ADC, three Timers, and Interrupts.

2 USER MANUAL

The user interface relies on a combination of analog and digital inputs, with real-time feedback provided by the LED matrix (and two debugging LEDs).

- **Playback Control:** Click the joystick to toggle between Playing and Paused.
- **Tempo Adjustment:** Push the joystick Up or Down to increase or decrease the playback speed (60 to 200 BPM).
- **Step Navigation:** Push the joystick Left or Right to manually move the active step cursor across the 32-step grid (Useful when paused).
- **Inputting Notes:** The top 12 keys of the matrix keypad correspond to musical notes. Pressing a key overwrites the currently selected step with that note, with the current editing octave. Bottom-left key ('1') is the note 'C', top-right key ('F') is the note 'B'.
- **Rests / Mutes:** Pressing the 'C' key mutes/clears the current step, creating a rhythmic rest.
- **Octave Shift:** Press the 'A' (decrease) or '0' (increase) keys to shift the editing octave up or down (1 to 4).
- **Display:** The LED matrix provides real-time visualization. It shows the entire 32-step note sequence, and it features a dedicated sector utilizing a custom 3x4 pixel font to display the current BPM and editing octave, while the remaining LEDs indicate the active/current step (and a separator line) (See figure 1).

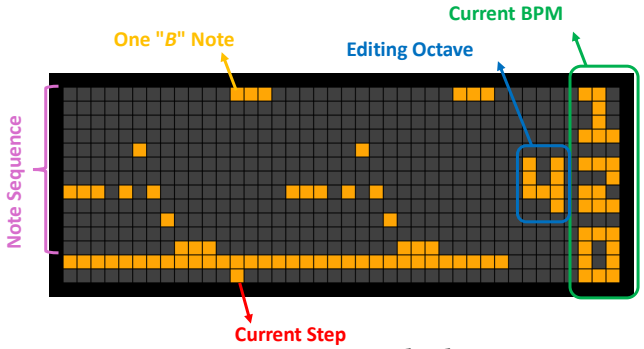

Figure 1: LED matrix display

Teaching team: Maarten Kuijk, Tuur Bruneel and Ferre Schelkens .

Microcontroller Project, Sensors and Microsystem Electronics - 009087, VUB (Bruface) 2026.

3 ARCHITECTURE

The codebase is structured around a foreground-background architecture with multiple subsystems. The main loop acts as the foreground, continuously polling user inputs and updating states, while hardware timers handle time-critical background tasks via Interrupt Service Routines (ISRs) and one Clear Timer on Compare Match (CTC) mode which toggles a pin in hardware.

3.1 Schematic

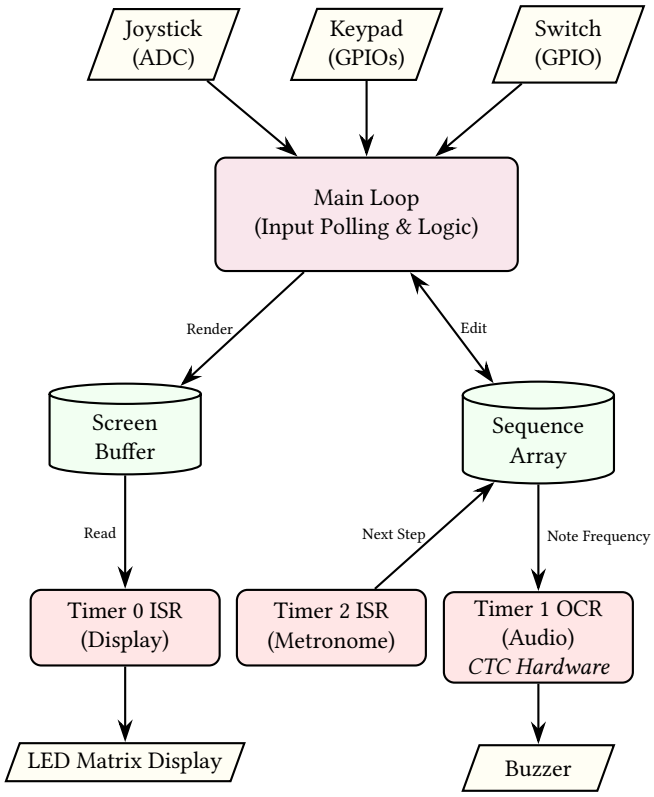

Figure 2: Block diagram of the hardware and software subsystem interactions.

3.2 Subsystem Interaction

- **Main Loop:**
  Continuously scans the ADC (joystick) and matrix keypad. When an input is detected, it applies state changes (e.g., updating the BPM variable, or modifying the sequence array).
- **Display Engine (Timer 0 OVF):**
  Runs independently in the background. It reads the shared screen buffer and handles the row-by-row hardware multiplexing required to keep the LED matrix illuminated without visible flickering.

- **Metronome (Timer 2 OVF):**
  Acts as the system clock. It increments a tick counter every 0.1ms and compares it against the user-defined tempo (converted to a delay). Upon a match, it advances the step pointer and commands the synthesizer to play the next note.
- **Synthesizer (Timer 1 OCR):**
  Generates the physical audio waveform directly in hardware. By utilizing Clear Timer on Compare Match (CTC) mode, it is configured such that the timer automatically toggles the physical output pin (OC1A) at the exact corresponding frequency of the active note (via the Output Compare Register, OCR). This produces the audio square wave entirely in hardware, requiring zero CPU overhead and no ISR.

3.3 Inputs

- **ADC Joystick:**
  The X and Y axes are read using the ATmega328P's internal ADC. The routine polls each channel and waits for conversion completion before comparing the 8-bit result against fixed high/low thresholds in order to get digital signals for the four directions.
- **Matrix Keypad:**
  The 4x4 keypad is scanned using the two-step polling method. The data direction registers and port outputs are rapidly inverted to switch rows and columns between inputs and outputs, identifying the exact matrix intersection of a pressed key.
- **Switch:**
  Disables the buzzer sound by skipping the `Play_Note` routine, to not annoy colleagues nearby, but still use the sequencer as normal.
- **Edge Detection:**
  All user inputs utilize dedicated SRAM state variables to track previous states. This prevents single presses from triggering multiple rapid executions and acts as a software debounce.

3.4 Main Data Structures

The project relies on specific memory layouts to ensure the microcontroller can process audio and graphics without lagging.

- **`Screen_Buffer` (SRAM):**
  A 560-byte array representing the LED matrix. It is intentionally *unpacked* (using one full byte per pixel instead of bit-packing). *Why:* While this consumes more SRAM, it drastically reduces the CPU cycles required inside the Timer 0 ISR, trading memory space (which we have plenty of) for execution speed to guarantee a high screen refresh rate, as well as easier code writing.
- **`Sequence` (SRAM):**
  A 32-byte array storing the user's song. Each index represents a step in time, and the value stores a note lookup index (or `0xFF` for a rest). *Why:* This 1-to-1 mapping makes stepping forward/backward computationally trivial.
- **System State & Debounce Variables (SRAM):**
  A collection of single-byte global variables (e.g., `Current_Step`, `Current_BPM`, `Tick_Counter`, etc) and input edge detection bytes. *Why:* Because the system relies on concurrent interrupts (like Timer 2 advancing the step while the Main Loop reads the joystick), these variables act as a shared state context, ensuring smooth software debouncing for buttons and preventing race conditions during playback.
- **`Note_Table` (Flash):**
  A pre-calculated array of timer delay values corresponding to musical frequencies. *Why:* The ATmega328P lacks a floating-point unit; using a LUT in flash allows instant pitch generation without computationally expensive math operations.
- **`Font3x4` (Flash):**
  A bit-packed lookup table containing the custom 3x4 pixel graphics for digits 0-9 and a blank character. *Why:* Bit-packing the font in ROM saves SRAM space. When the BPM or Octave needs updating, the code decodes these bits and draws them to the `Screen_Buffer`.

4 FEATURES SUMMARY

- Real-time adjustable metronome (60 to 200 BPM) without playback interruption.
- 32-step programmable sequence tracking.
- Unpacked display memory buffer with direct memory-address calculation for fast screen wiping and rendering.
- Easy-to-use user inputs (joystick, keypad and switch).
- Sound output enable/disable switch (to play without annoying colleagues around you).
- Play/pause function to easily edit individual steps.

5 CONCLUSION

Developing this project provided deep practical insight into the AVR architecture. Relying strictly on assembly language highlighted the critical importance of register preservation during interrupts and function calls, memory layout optimization, and hardware constraints. Successfully combining ADC polling, Timer-driven multiplexing, and direct port manipulation into a single concurrent system demonstrated the robust capabilities of bare-metal embedded design. In the end, I managed to create a fully-featured 32-step sequencer on the limited ATmega328P platform.

6 ACKNOWLEDGMENT

Thank you to Salman Houdaibi, Rodolphe Valicon, and Alexis Bollengier for their help during brainstorming and debugging sessions.

Disclosure of AI/LLM use: LLMs were used for help debugging some code, for help in language formulation of this report (not being a native English speaker), and for extra explanations for harder or niche concepts of AVR assembly. LLMs were absolutely **not** used for the writing of code.

APPENDIX

A video of the device operating can be viewed through this link: https://youtu.be/Ndic62PA8RU.

The entire codebase is available in the project's Github repository: https://github.com/LucasPlacentino/AVRasm-Sequencer.

BIBLIOGRAPHY

2008. AVR® Instruction Set Manual. https://ww1.microchip.com/downloads/aemDocuments/documents/MCU08/ProductDocuments/ReferenceManuals/AVR-InstructionSet-Manual-DS40002198.pdf

2009. ATmega328P 8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash Datasheet. https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf